Beyond the GPU

A Microarchitectural Analysis of LLM Tool-Calling Kernels

Kunpeng Zhang / TJ Grewal / Fazal Rehman

Simon Fraser University

What happens when the CPU becomes the bottleneck for AI?

Outline

Part I – TJ Grewal

1. Motivation & The Agentic Shift
2. Background: ReAct & Tool-Calling
3. Research Questions

Part II – Kunpeng Zhang

4. Decoupled System Architecture
5. PAPI Telemetry Integration
6. C++ Kernels & Hardware Optimizations

Part III – Fazal Rehman

7. Quantifying the Agentic Tax
8. Multi-Step Compounding Overhead
9. The Masking Effect
10. Limitations & Future Work
11. Conclusion

The Shift: From Chatbot to Autonomous Agent

- LLMs are rapidly evolving from **static chat interfaces** into **autonomous agents**.
- Modern agents *reason*, *call tools*, *parse results*, and *iterate*.
- Historical optimization focus: **GPU forward-pass**
  - Time-To-First-Token (TTFT)
  - Inter-token latency
- But agents spend most of their time **outside** the GPU...

Key Insight

The execution bottleneck is shifting **from the accelerator back to the host CPU**.

The Gap

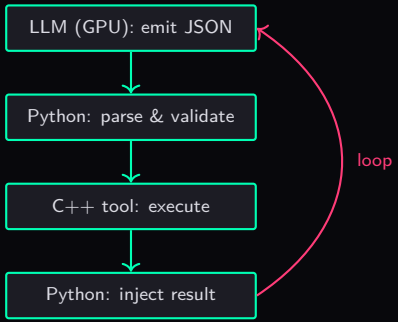
Current literature treats the tool-execution pipeline as a *black box*.

What is “Tool Calling”? – The ReAct Framework

ReAct = Reasoning + Acting

1. LLM reads the user prompt
2. LLM emits a **structured JSON** tool call
3. Python orchestrator **parses** the JSON
4. Orchestrator **executes** a native subprocess
5. `stdout` is **injected back** into context
6. Repeat until the task is complete

This loop is strictly serial.



Architectural Divergence: ML vs. Datacenter Workloads

GPU-Friendly (Matrix Math)

- Dense matrix multiplication
- Massively parallel
- Predictable memory access
- High arithmetic intensity

Agentic Tool Workloads

- JSON parsing, DB lookups, FS walks
- Serial, branch-heavy
- Pointer chasing $\rightarrow$ LLC misses
- Behave like **datacenter RPCs**

Agent tools are fundamentally hostile to GPU architectures.

Research Questions & Contributions

Research Questions

1. How much CPU overhead does the LLM orchestration layer actually impose?
2. Do classical hardware optimizations (SIMD, prefetching, OpenMP) help agentic tools?
3. Does kernel-level efficiency propagate to end-to-end latency?

Our three contributions:

1. **Quantification of the Agentic Tax:** up to **69.5**× slowdown on lightweight tools.
2. **Microarchitectural characterization** of compute-, memory-, and I/O-bound agent tools.
3. **Identification of the Masking Effect:** kernel-level wins rendered statistically invisible at the system level.

Decoupled System Architecture

Goal: Isolate microarchitectural overhead *without* cross-contamination between Python and C++.

Three Decoupled Components

1. **Orchestrator (Python / FastAPI)**: prompt assembly, inference, JSON routing.
2. **Execution Kernels (C++)**: compiled native binaries for compute / memory / I/O.
3. **Static Workloads**: pre-generated deterministic stress payloads.

Why Decouple?

- Isolates C++ from Python GIL
- Avoids Python memory-management noise
- Enables pristine profiling

Model: Qwen2.5-7B-Instruct (FP16 + SDPA), `do_sample=False`

PAPI: Hardware Performance Counters

Performance API → direct access to the Linux kernel's Performance Monitoring Unit (PMU).

Four hardware events tracked

- `PAPI_TOT_CYC` – total cycles
- `PAPI_TOT_INS` – total instructions
- `PAPI_BR_MSP` – branch mispredictions
- `PAPI_L3_TCM` – L3 cache misses

Windowing strategy:

- **CPU Window 1:** prompt assembly + tokenization
- **CPU Window 2:** decoding + JSON extraction

Testbed

Intel Xeon Gold 6242R @ 3.10 GHz – isolated, predictable cycle-to-latency.

Hardware EventSets guarded in `try...finally` to avoid PMU starvation.

Three C++ Tool Kernels

Compute-Bound

AST Math Evaluator

- Nested math expressions
- Recursive parser
- Stresses **branch prediction**
- Tests instruction throughput

Memory-Bound

SQLite DB Scan

- Forced full-table scan
- `LIKE` bypasses indexes
- Thrashes **LLC (L3)**
- Classic memory wall

I/O-Bound

Directory Walker

- Recursive FS traversal
- OS metadata queries
- Stresses **syscalls**
- Context-switch heavy

Each kernel manages its own PAPI profiling around the core workload.

The Control Baseline: `pipeline_mode="direct"`

Question: How do we prove the "Agentic Tax" exists?

Direct Baseline

- GPU and LLM **completely bypassed**
- Regex extracts target paths from prompt
- C++ kernels invoked directly
- Represents the theoretical "speed of light"

Full Agentic Loop

Qwen-2.5-7B autonomously:

- Parses prompt
- Emits JSON
- Executes tool
- Re-injects result

Result: Subtracting Direct from Full Agentic gives us the **exact cost of orchestration**.

Direct vs. Agentic
= the Agentic Tax

Hardware Optimizations Applied

1. SIMD Vectorization (AVX2)

Applied to the AST math evaluator. 256-bit wide SIMD registers intended to accelerate numerical parsing.

2. Software Prefetching

`__builtin_prefetch` with lookahead distance of 32 elements. Latency hiding for cold-cache DB scans.

3. OpenMP Concurrency

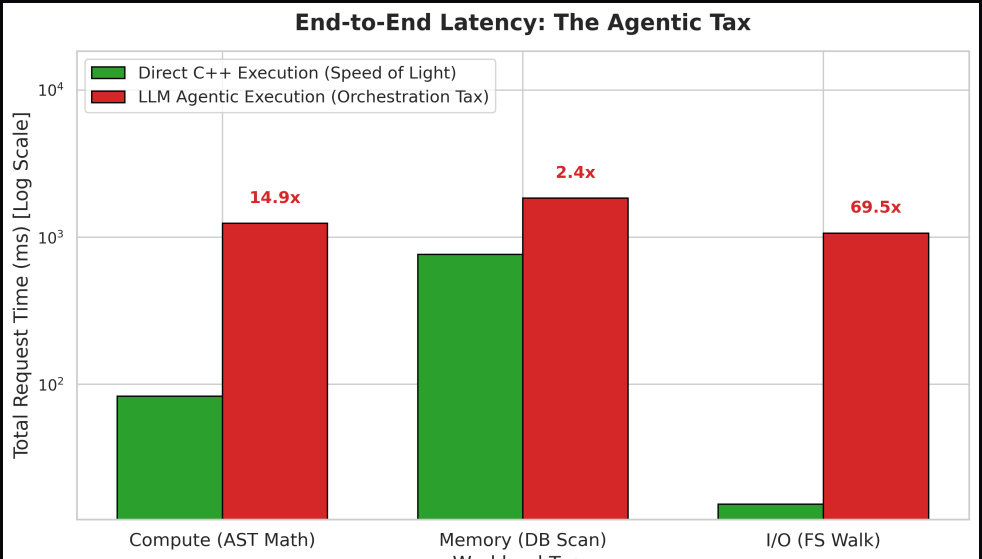
Task-parallel directory walker. Overlaps I/O-bound system calls across threads.

Compiler Flags

```
-O3 -march=native -mavx2
-O3 -march=native -fopenmp
```

Do these optimizations matter in an agentic context?

Quantifying the Agentic Tax (End-to-End Latency)



The Agentic Tax is a Fixed-Cost Penalty

- **I/O walker:** 69.5× slowdown
  Native: ∼15 ms → Agentic: ∼1 s
- **AST math:** 14.9× slowdown
- **DB scan:** only 2.4× slowdown
  ∼764 ms native, Agentic tax amortized

Key Finding

The tax is **not** proportional; it's a **massive fixed cost** that disproportionately punishes lightweight tools.

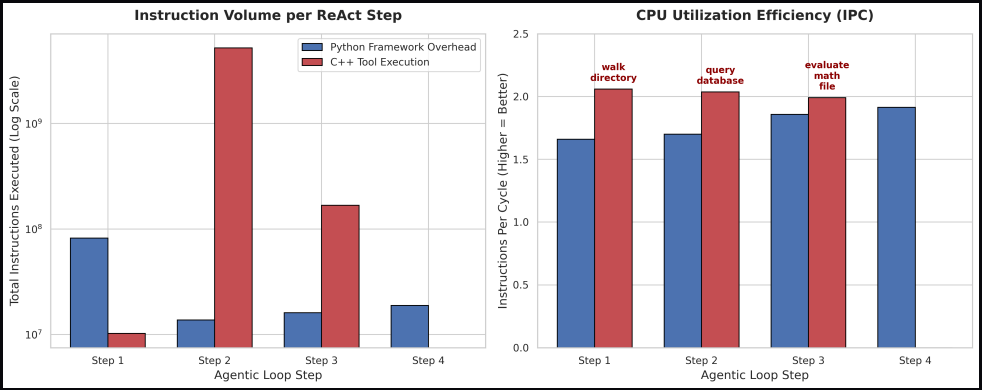
Amdahl Saves the DB scan

The heavy 2.5 billion-cycle DB workload is so long that the orchestration overhead becomes a *proportionally* smaller slice.

For most tools, the framework dominates execution time.

Compounding Overhead in Multi-Step Workflows

"Gauntlet" prompt: execute I/O walk → DB query → math evaluation in **one context window**.



(Left) Framework vs. tool instruction volume. (Right) IPC gap between Python and native C++.

Two Phenomena Revealed by Multi-Step Telemetry

1. Framework: Cold Start + Drift

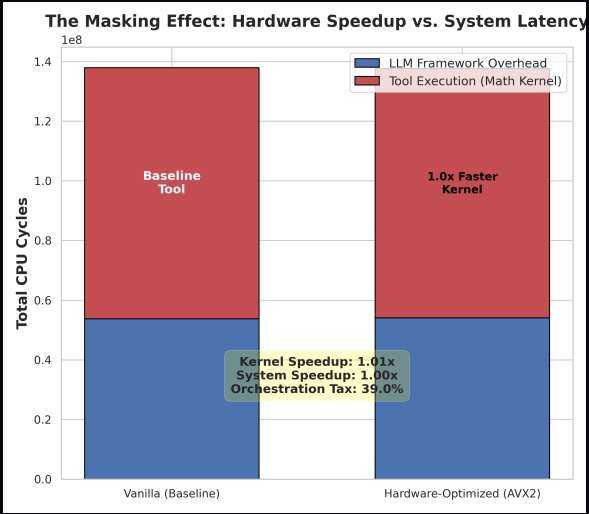
- **Step 1:** ~82M instructions
  one-time JSON schema load
- **Steps 2–4:** baseline ~13.7M
  grows linearly to 18.8M
- Cause: **stateless re-serialization** of the growing context window
- Framework IPC rises 1.65 → 1.91

2. Tools: Payload-Dictated

- C++ metrics independent of step count
- Instructions: **10M** (I/O) → **5B** (DB)
- Kernels near ~2.0 IPC (max throughput)
- DB scan IPC = 2.03 (prefetcher wins)
- AST IPC = 1.99 (branch mispredicts)

LLM coordination overhead is a compounding tax that scales with autonomy.

Optimization 1: The Vectorization Wall (AVX2)



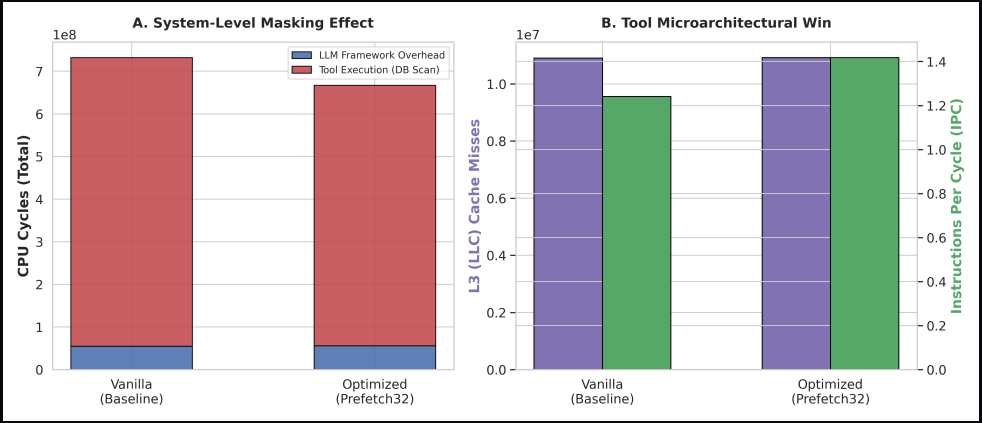
Hypothesis: AVX2 (256-bit SIMD) should accelerate nested math parsing.

Result: FAILED

- IPC stagnant at ≈ 1.99
- Cycles: $84.1\mathrm{M} \rightarrow 83.6\mathrm{M}$
- Only **0.6%** improvement

Why: Recursive, branch-heavy, pointer-chasing AST parsing **defeats auto-vectorization**.

Optimization 2: Software Prefetching Wins. . . at the Kernel



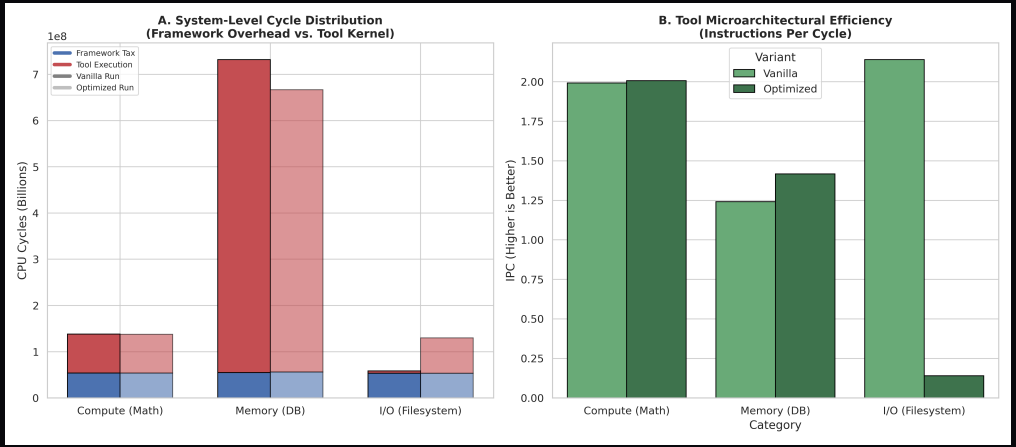
Microarchitectural Victory

- **IPC: +14.2%** (1.241 → 1.417)

System-Level Reality

- Total request ≈ 6.3 s

The Masking Effect – System-Level View



Kernel-level wins are rendered statistically invisible to the end user.

Limitations

1. PAPI Profiling Artifact (OpenMP)

- High-level PAPI tracks **only the calling thread**
- Master thread spin-waits on OpenMP barrier $\rightarrow$ cycles accumulate but instructions don't
- Reported IPC plummeted to **0.14** – a profiling blind spot, not a real regression
- **Fix:** thread-local PAPI EventSets for future multi-threaded work

2. OpenMP Negative Scaling

- Parallel FS walker used 15$\times$ *more* CPU cycles than the serial baseline
- Root cause: `std::atomic` global counter $\rightarrow$ L1 cache-line contention
- Plus OS-kernel serialization of small-directory disk reads

3. Static Workloads

Conclusion: Rethinking Agentic Systems

Headline Findings

- Lightweight agent tools suffer up to $\mathbf{69.5}\times$ latency slowdown.
- Kernel IPC improvements of $\mathbf{14.2}\%$ translate to $< \mathbf{1}\%$ end-to-end speedup.
- This is the **Masking Effect**: framework bloat swallows hardware victories.

Future Work:

1. **High-throughput inference engines:** continuous batching + PagedAttention (e.g., vLLM).
2. **Compiled orchestrators:** migrate Python $\rightarrow$ Rust/C++; use SIMD JSON parsers (`orjson`, `simdjson`).
3. **Binary protocols:** fine-tune LLMs to emit Apache Arrow instead of JSON strings.

Until the orchestration layer is treated as a high-performance data plane, the promise of agentic AI will remain masked by its own framework.

Thank You

Beyond the GPU:

A Microarchitectural Analysis of LLM Tool-Calling Kernels

Kunpeng Zhang · TJ Grewal · Fazal Rehman

Simon Fraser University

github.com/AndyZzzZzzZzz/beyond-gpu-kernels